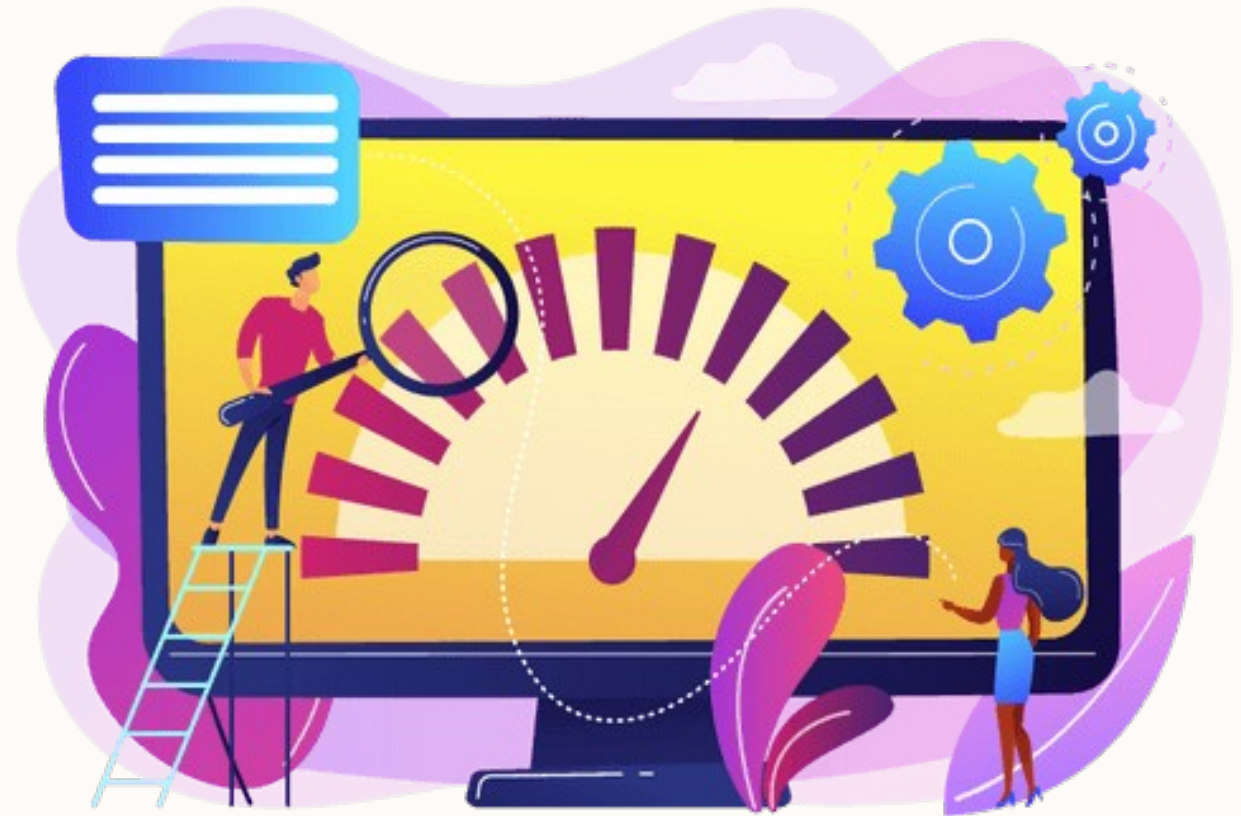


Pruebas no funcionales

Rendimiento, resistencia,
crecimiento y adaptación

Presentación de clase



**¿Cómo se comporta el sistema
cuando cambia el contexto de uso?**

Agenda

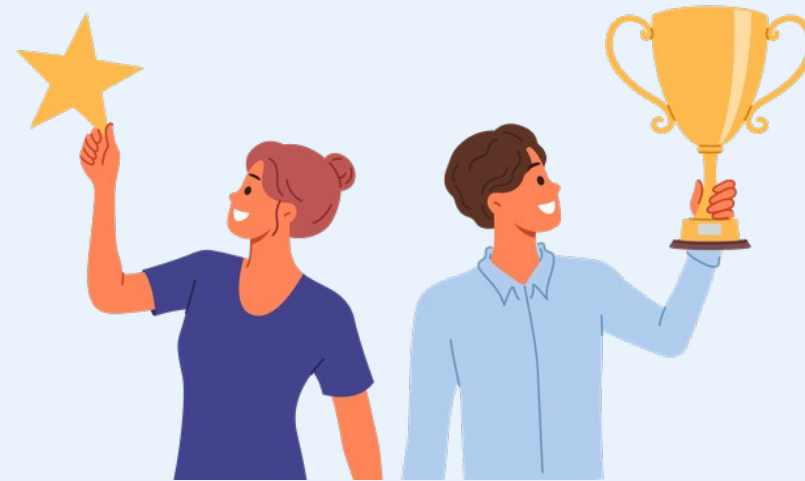
- | | | | | | |
|----|----|------------------------------------|----|----|--------------------------------|
| 01 | —— | Dudas del proyecto | 08 | —— | Pruebas de carga |
| 02 | —— | Qué son las pruebas no funcionales | 09 | —— | Recurso para practicar: Locust |
| 03 | —— | Ejemplo práctico | 10 | —— | Pruebas de estrés |
| 04 | —— | Conceptos y principios | 11 | —— | Pruebas de escalabilidad |
| 05 | —— | Diseñar una prueba medible | 12 | —— | Pruebas de portabilidad |
| 06 | —— | Importancia y aplicabilidad | 13 | —— | Mini caso de aplicación |
| 07 | —— | Pruebas dentro de la arquitectura | | | |

Del concepto a la práctica

Antes del contenido

Dudas del proyecto

En este momento hablaremos de las dudas, decisiones o puntos pendientes del proyecto.



**Después de este espacio,
retomamos el tema de la clase.**

¿Qué son las pruebas no funcionales?

Evalúan la calidad con la que el sistema trabaja, no solamente las funciones que ofrece.

Analizan el comportamiento del sistema bajo distintas condiciones de uso y permiten comprobar si cumple criterios como rendimiento, capacidad de respuesta, estabilidad o adaptación.

Pregunta guía

¿El sistema sigue siendo útil y confiable cuando cambia la forma de utilizarlo?



Calidad observable

Ejemplo práctico: una tienda en línea

La función puede estar correcta y aun así la experiencia puede fallar.

Prueba funcional

- ¿La búsqueda encuentra el producto?
- ¿El pago registra la compra?

Resultado: la función responde correctamente.

Pruebas no funcionales

- ¿Responde a tiempo?
- ¿Soporta muchos usuarios?
- ¿Se recupera si aumenta el tráfico?
- ¿Funciona en móvil y otros navegadores?

Resultado: la experiencia mantiene una calidad aceptable.

Conceptos y principios

Conceptos clave

Atributo	Característica de calidad que se puede observar y medir.
Escenario	Condición de uso, métrica y criterio que definen la prueba.
Métrica	Dato que permite comparar el comportamiento con el objetivo.

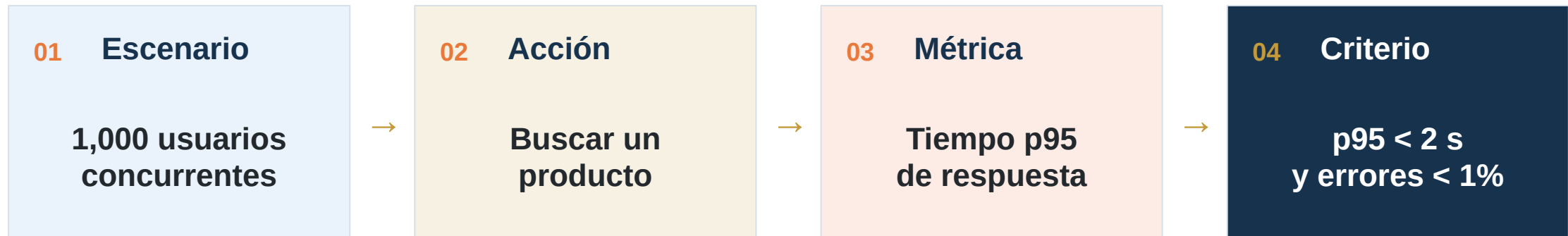
Principios

- 01 Medir con datos**
- 02** Usar condiciones realistas
- 03** Repetir y comparar
- 04** Automatizar cuando sea posible

Una prueba útil deja evidencia para decidir.

De la pregunta al criterio de aceptación

Una prueba se vuelve útil cuando la pregunta termina en un criterio que podemos comprobar.

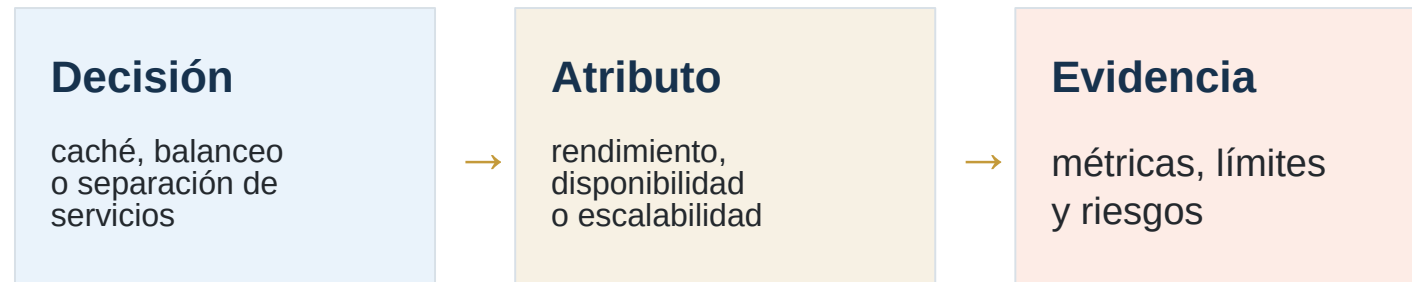


p95 significa que el 95% de las solicitudes responde en ese tiempo o menos.

La métrica conecta la experiencia con una decisión técnica.

Importancia y aplicabilidad en la arquitectura

En una arquitectura de software, las decisiones técnicas condicionan la calidad que el usuario percibe.



Aportan valor porque:

Previenen cuellos de botella antes de producción.

Validan que la arquitectura soporte la carga prevista.



¿Dónde se observa la calidad?

Cada parte de la arquitectura puede introducir un límite distinto.



La evidencia ayuda a ubicar el problema antes de cambiar la solución.

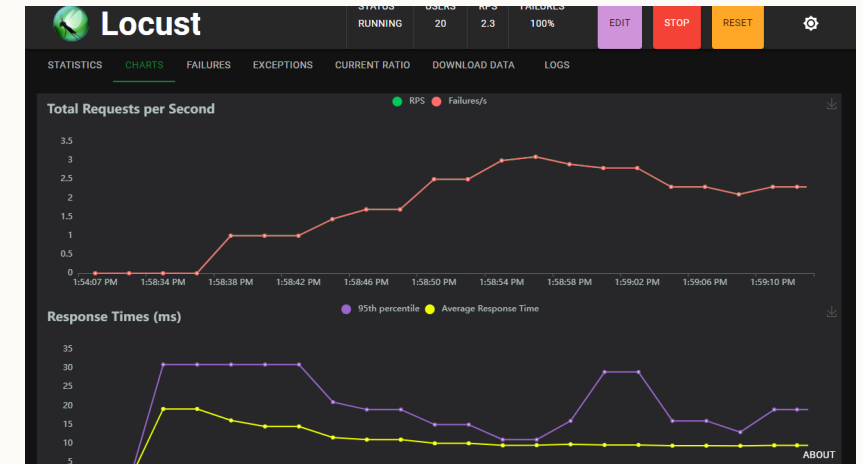
Pruebas de carga

Simulan la carga esperada para comprobar si el sistema mantiene un desempeño aceptable.

Se observan principalmente:

- tiempo de respuesta
- solicitudes procesadas
- uso de CPU, memoria y red
- estabilidad durante la operación

Ejemplo ilustrativo: 1,000 usuarios concurrentes realizando búsquedas durante 10 minutos.



Pregunta guía

¿Funciona bien en el escenario normal?

Locust permite simular usuarios y observar el comportamiento de una aplicación web.



- 01 Definir**
escenarios de usuario en Python
- 02 Ejecutar**
usuarios virtuales con una carga definida
- 03 Observar**
solicitudes por segundo y tiempos de respuesta

Recurso sugerido para experimentar, no un requisito del tema.

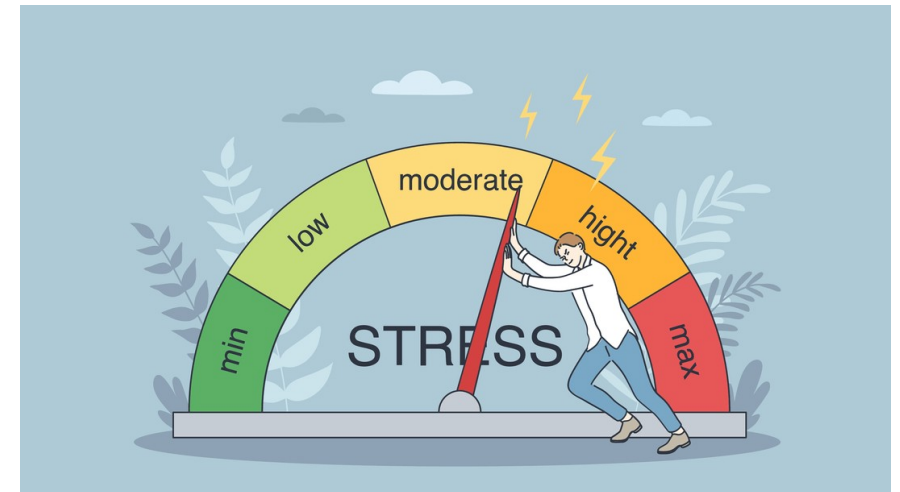
Pruebas de estrés

Llevan el sistema más allá de la carga esperada para encontrar su límite y observar su recuperación.

Buscan responder tres preguntas:

1. ¿En qué punto comienza a fallar?
2. ¿El fallo ocurre de forma controlada?
3. ¿Cuánto tarda en recuperarse?

Ejemplo: someter una tienda en línea a un tráfico 10 veces superior al máximo esperado.



Carga: escenario esperado

Estrés: sobrecarga y recuperación

Pruebas de escalabilidad

Evalúan si el sistema puede crecer o reducirse cuando aumentan los usuarios, datos o transacciones.

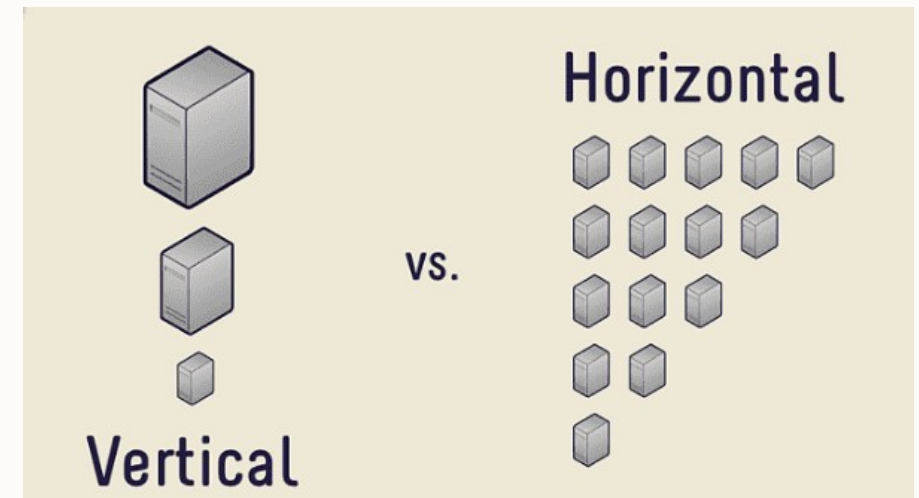
El punto clave es mantener un rendimiento adecuado mientras cambia la demanda.

Vertical

Aumentar la capacidad de un servidor: CPU, memoria o almacenamiento.

Horizontal

Agregar nodos o servidores y repartir el trabajo entre ellos.



Pruebas de portabilidad

Comprueban si el sistema puede funcionar correctamente al trasladarse a otros entornos.

Se pueden revisar cambios de:

- sistema operativo
- navegador o versión
- dispositivo y tamaño de pantalla
- servidor, nube o configuración

Ejemplo: validar el mismo sistema en Windows y Linux, en Chrome y Firefox, y en computadora y móvil.



Misma solución,
entornos distintos

Mini caso: elige la prueba correcta

Lee cada situación y relaciona la pregunta con el tipo de prueba.

A Una jornada normal con 1,000 usuarios **Carga**

B Un pico inesperado diez veces mayor **Estrés**

C Pasar de un servidor a cinco nodos **Escalabilidad**

D Usar el sistema en móvil y otro navegador **Portabilidad**

Pista

Pregunta qué cambia:

la cantidad de usuarios,
el límite,
la capacidad
o el entorno.